Laboratório Funcional · Aula 12

VINÍCIUS JIENTARA CABRAL · RA 24191501-2 · Turma C
Gerado em 16/10/2026, 20:39:39 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

1) def agora(): return datetime.now() — o resultado não depende só dos parâmetros: ela lê o relógio do sistema, então duas chamadas iguais devolvem valores diferentes.
2) def poe(lista, x): lista.append(x); return lista — tem efeito colateral: o append modifica a própria lista recebida como parâmetro (altera algo fora da função), então a chamada não pode ser trocada pelo valor que devolveu.

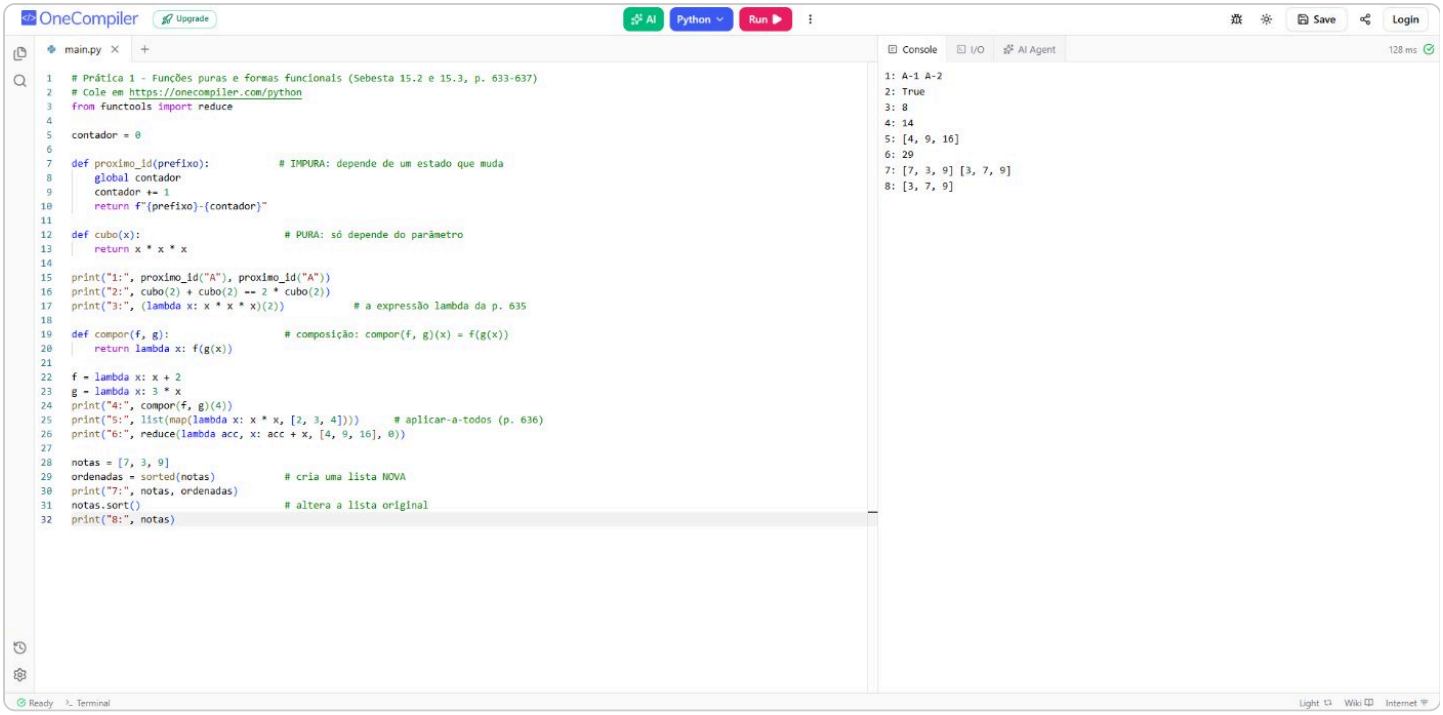
Prática 1: Funções puras e formas funcionais

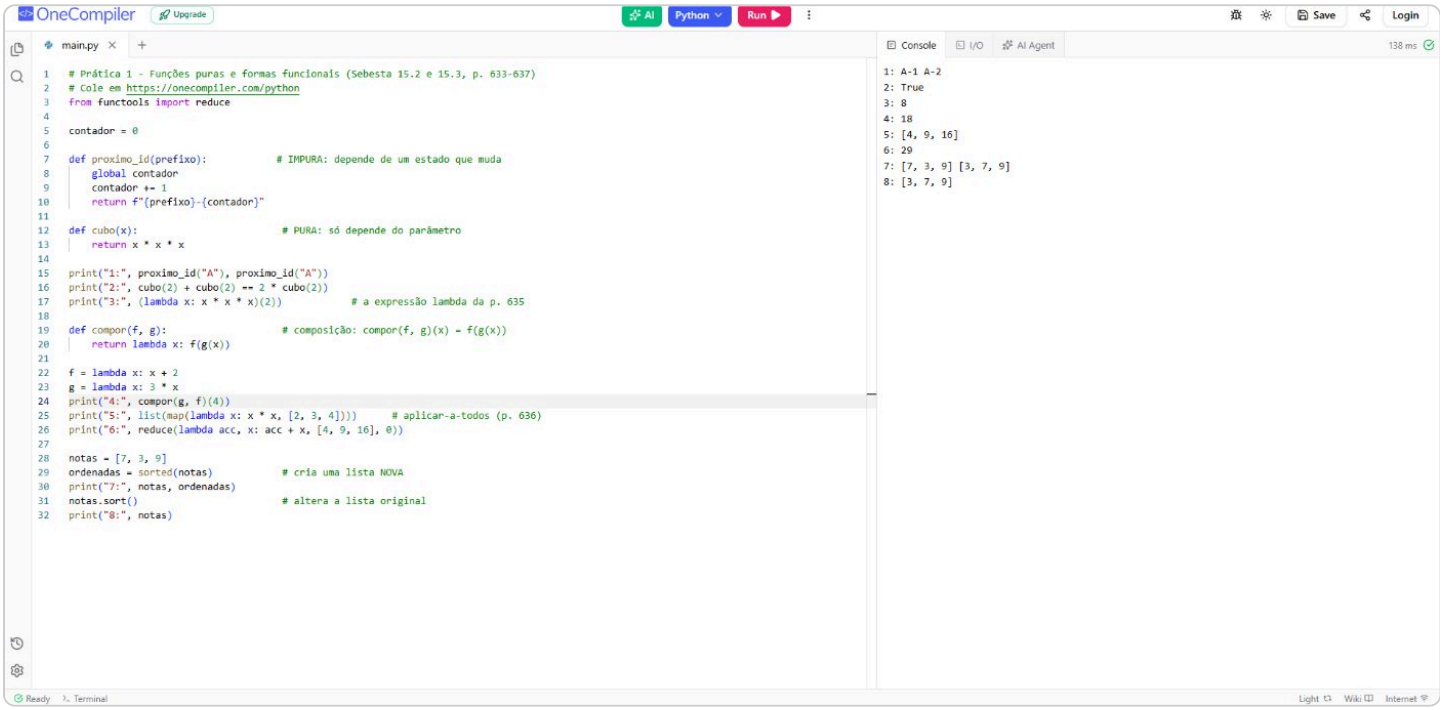
Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Linha 1: A-1 A-2 (o contador sobe a cada chamada).
Linha 4: 14.
Linha 7: [7, 3, 9] [3, 7, 9] (sorted cria uma lista nova).

Saídas (original e com compor(g, f))

não respondido





1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

Porque proximo_id depende de uma variável global (contador) que ela mesma altera a cada chamada: o resultado não depende só do parâmetro. A primeira chamada devolveu A-1 e a segunda A-2. Isso quebra a transparência referencial e dificulta os testes (p. 637): para testar é preciso conhecer e controlar o estado global, o resultado depende da ordem e do número de chamadas anteriores, e a função não pode ser testada isoladamente.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

f(x)=x+2 e g(x)=3x. compor(f,g)(4)=f(g(4))=f(12)=14. compor(g,f)(4)=g(f(4))=g(6)=18. A composição não é comutativa: a ordem das funções muda o resultado.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) devolve uma lista nova ordenada e deixa notas intacta (linha 7: [7, 3, 9] e [3, 7, 9]). notas.sort() ordena a própria lista, alterando o original (linha 8: [3, 7, 9]), e devolve None. O sorted segue o estilo funcional: não tem efeito colateral, produz um valor novo em vez de modificar o existente.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

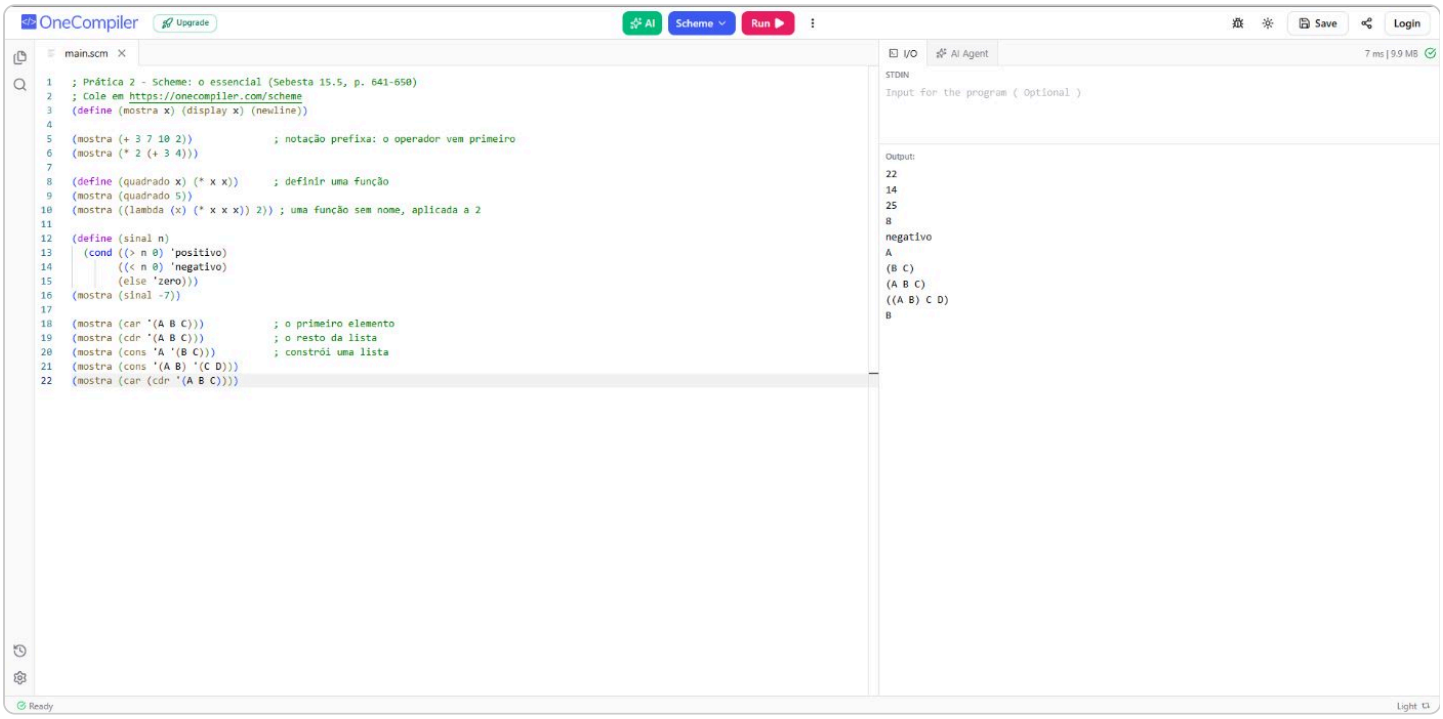
(B C)

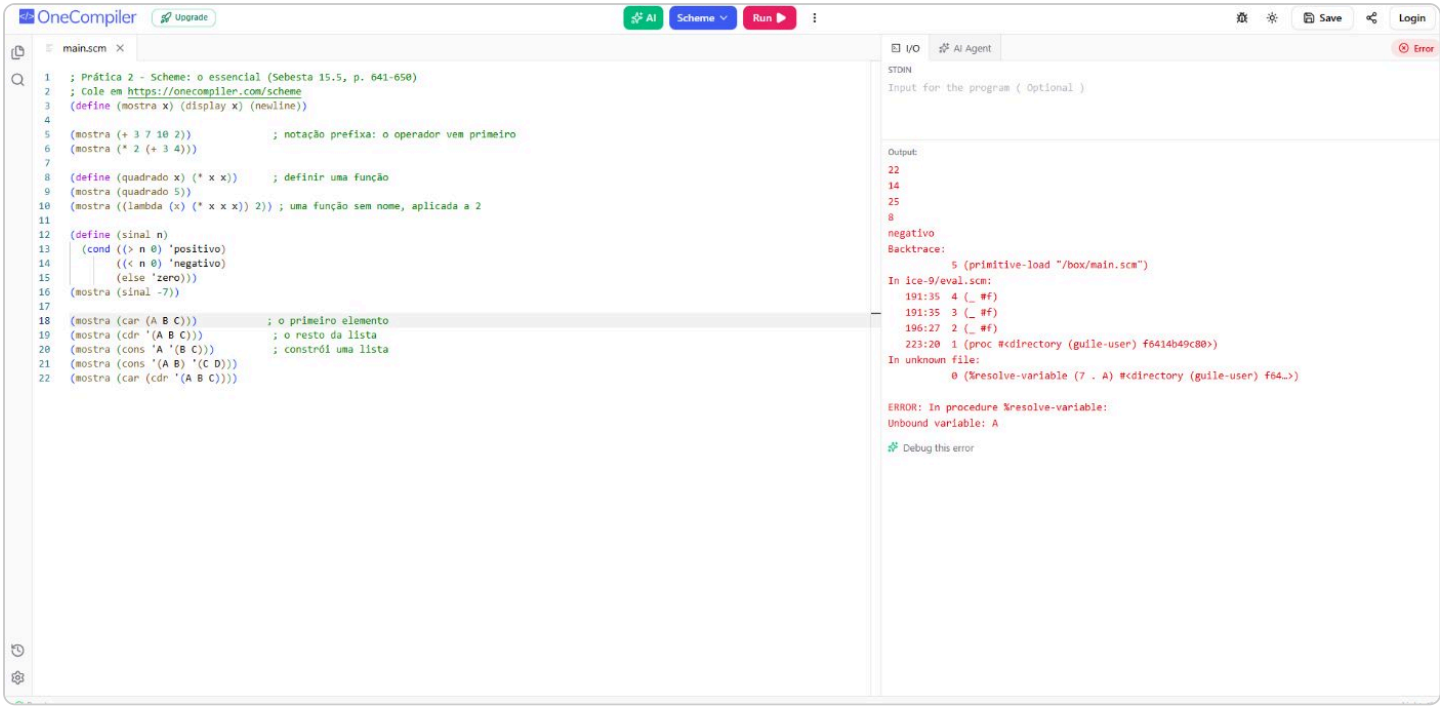
Previsão: (cons '(A B) '(C D))

((A B) CD)

Saídas (original e sem o apóstrofo)

não respondido





2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo é a abreviação de QUOTE: faz o Scheme tratar a lista como dado, sem avaliá-la. Sem ele, (A B C) é avaliada como uma chamada de função: o Scheme tenta chamar A (o primeiro elemento) com os parâmetros B e C, e dá erro porque A não está definida (Unbound variable: A).

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))
Resultado: C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

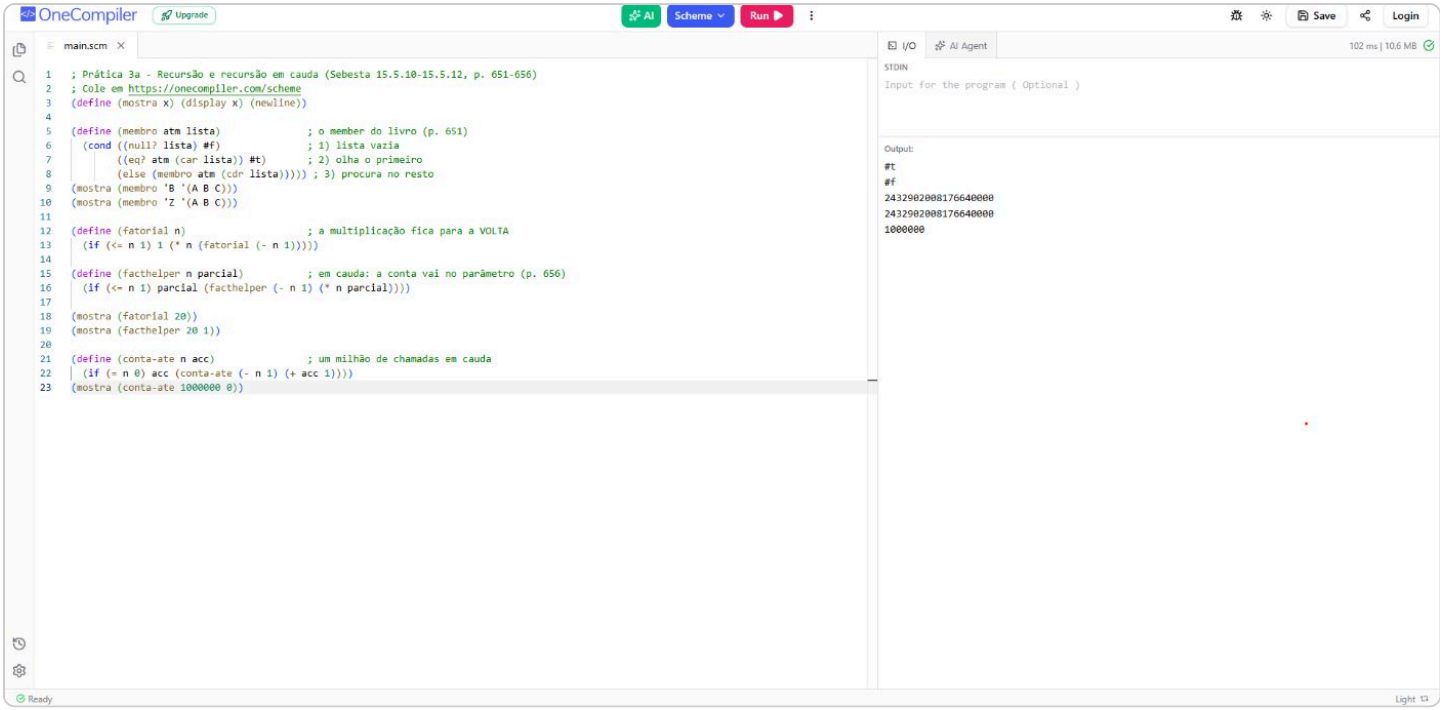
n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).
n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Scheme: sim, deve funcionar, porque a chamada recursiva é a última operação (em cauda) e o Scheme reaproveita o registro de ativação. Python: acho que não, vai estourar a pilha com RecursionError.

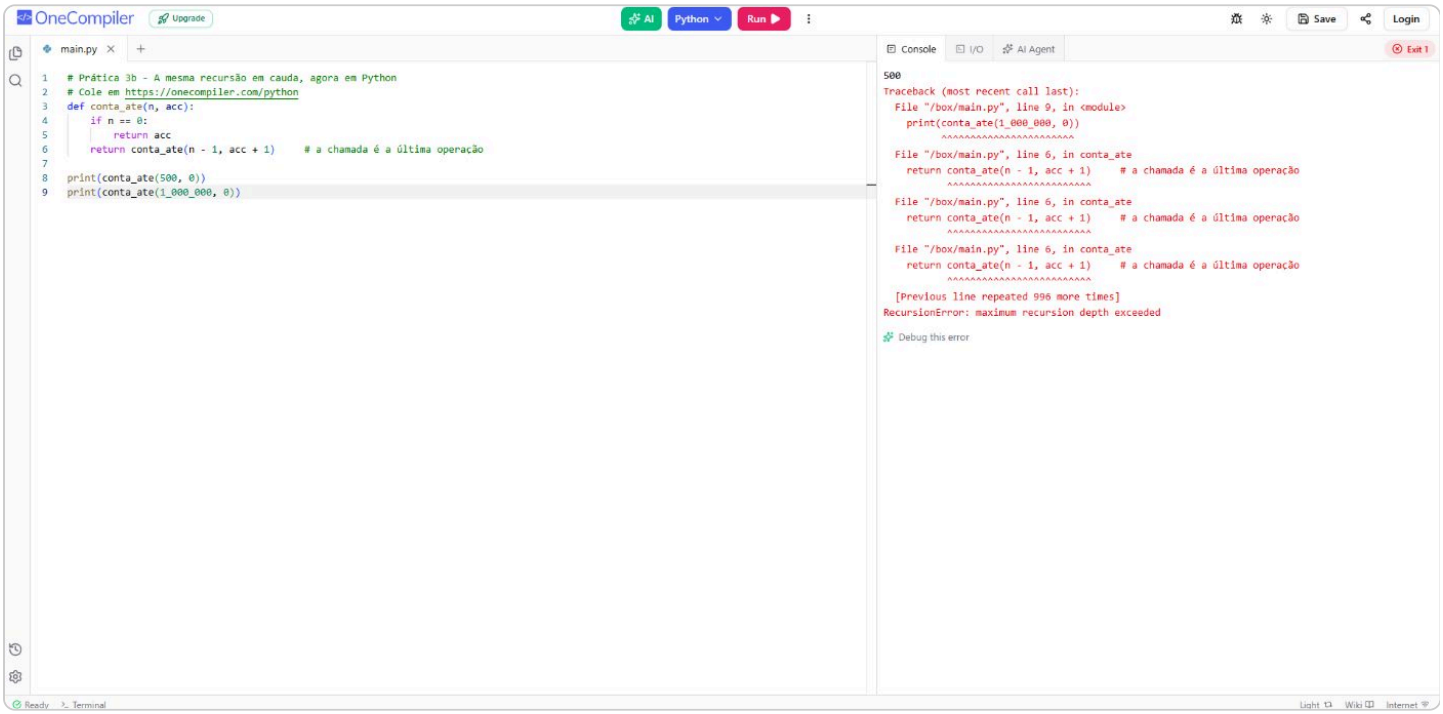
Saída do Scheme

não respondido



Saída do Python (as últimas linhas bastam)

não respondido



3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum cresce o número de multiplicações pendentes (e o tamanho da expressão): cada chamada fica esperando a seguinte voltar para multiplicar por n, então há um registro de ativação empilhado por chamada (com n = 4 chegou a 3 pendentes; com n = 6, a 5). Na recursão em cauda nada cresce: o resultado parcial vai no acumulador, nada fica esperando e a expressão tem tamanho praticamente constante, por isso o registro pode ser reaproveitado.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme faz a otimização da chamada em cauda: como conta-ate é recursiva em cauda, ele reaproveita o registro de ativação e a recursão ocupa espaço constante, como um laço (p. 656). O Python não faz essa otimização e limita a profundidade da pilha (cerca de 1000 chamadas), então cada chamada empilha um registro novo e, perto de 1000, dá RecursionError: maximum recursion depth exceeded, mesmo a função sendo em cauda.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3 ok
soma-quadrados: 14 ok
pares: (2 4 6) ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis)) (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi pares, porque tem três casos (lista vazia, primeiro par, primeiro ímpar) e é preciso decidir quando usar cons para reconstruir a lista e quando só seguir com o cdr. Em todas as funções, null? testa o caso base (lista vazia); car pega o primeiro elemento (em soma-quadrados ele é elevado ao quadrado; em pares, é testado com even?); cdr entra na chamada recursiva sobre o resto da lista. Em tamanho o car nem é usado, só null? e cdr.

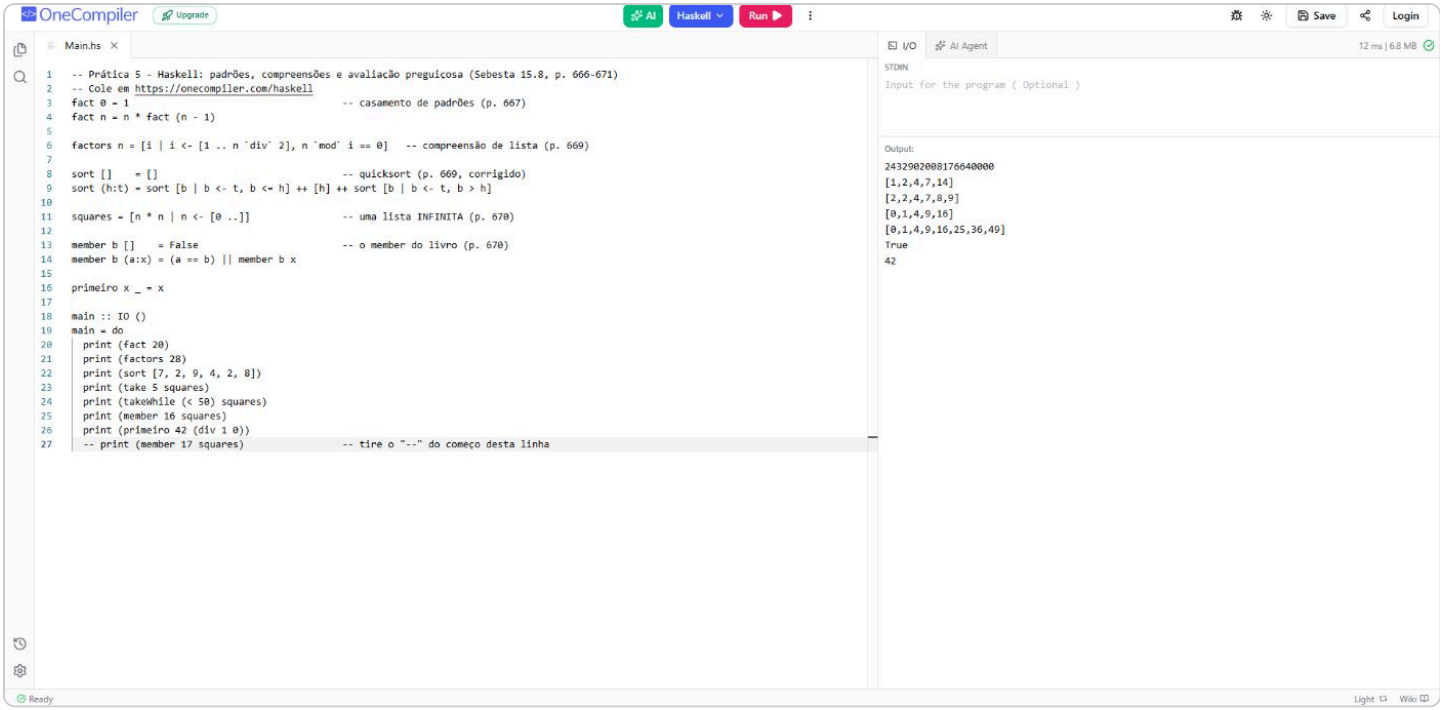
Prática 5: Haskell: padrões, compreensões e preguiça

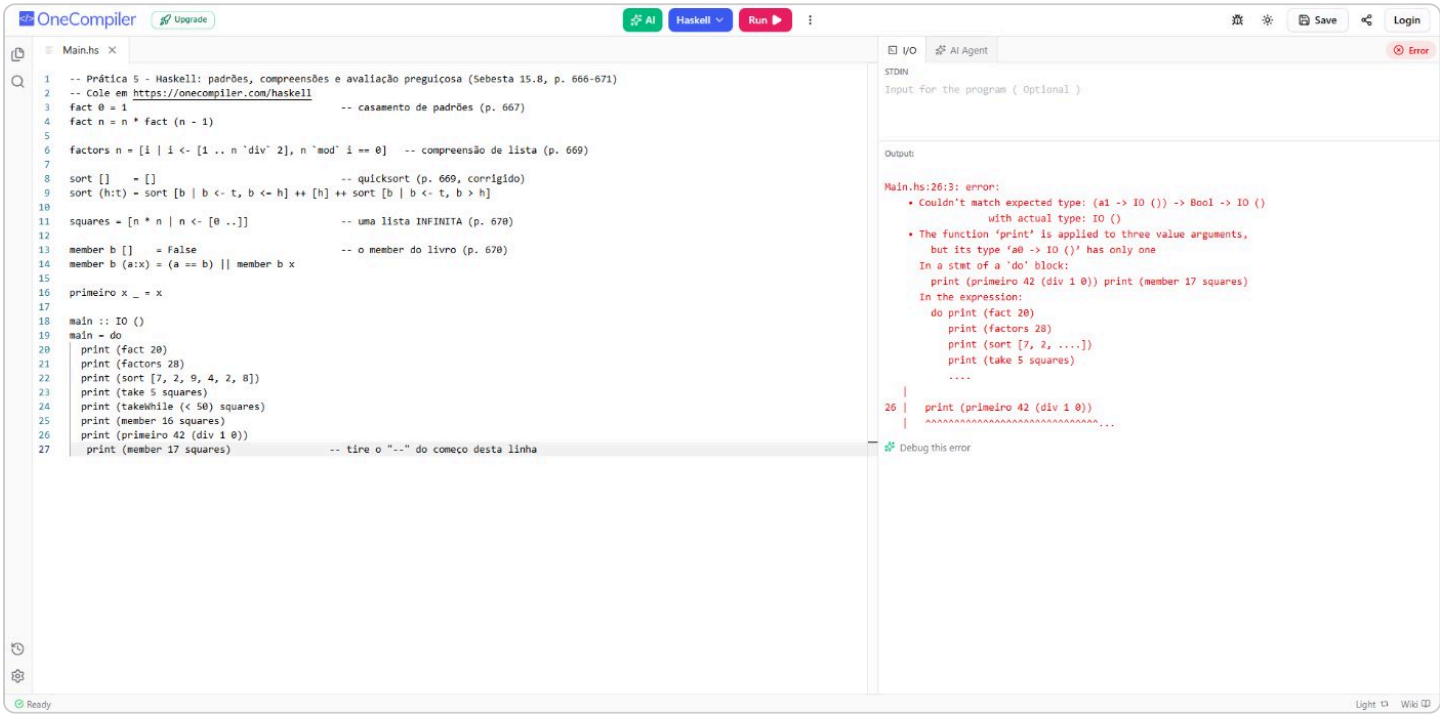
Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares imprime [0,1,4,9,16]. primeiro 42 (div 1 0) imprime 42: acho que não dá erro, porque o segundo parâmetro nunca é usado.

Saídas (original e com member 17)

não respondido





5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

Haskell usa avaliação preguiçosa: calcula apenas os primeiros cinco elementos solicitados por take, não a lista infinita inteira. primeiro ignora o segundo parâmetro; como div 1 0 não é necessário para obter o resultado, não é avaliado e não gera erro.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

member 17 squares não termina porque 17 não aparece na lista de quadrados e a busca continua para sempre. member 16 termina porque encontra 16. Como squares é crescente, a busca pode parar quando o elemento atual for maior que o procurado; por exemplo, member b (a:x) = if a > b then False else (a == b) || member b x.

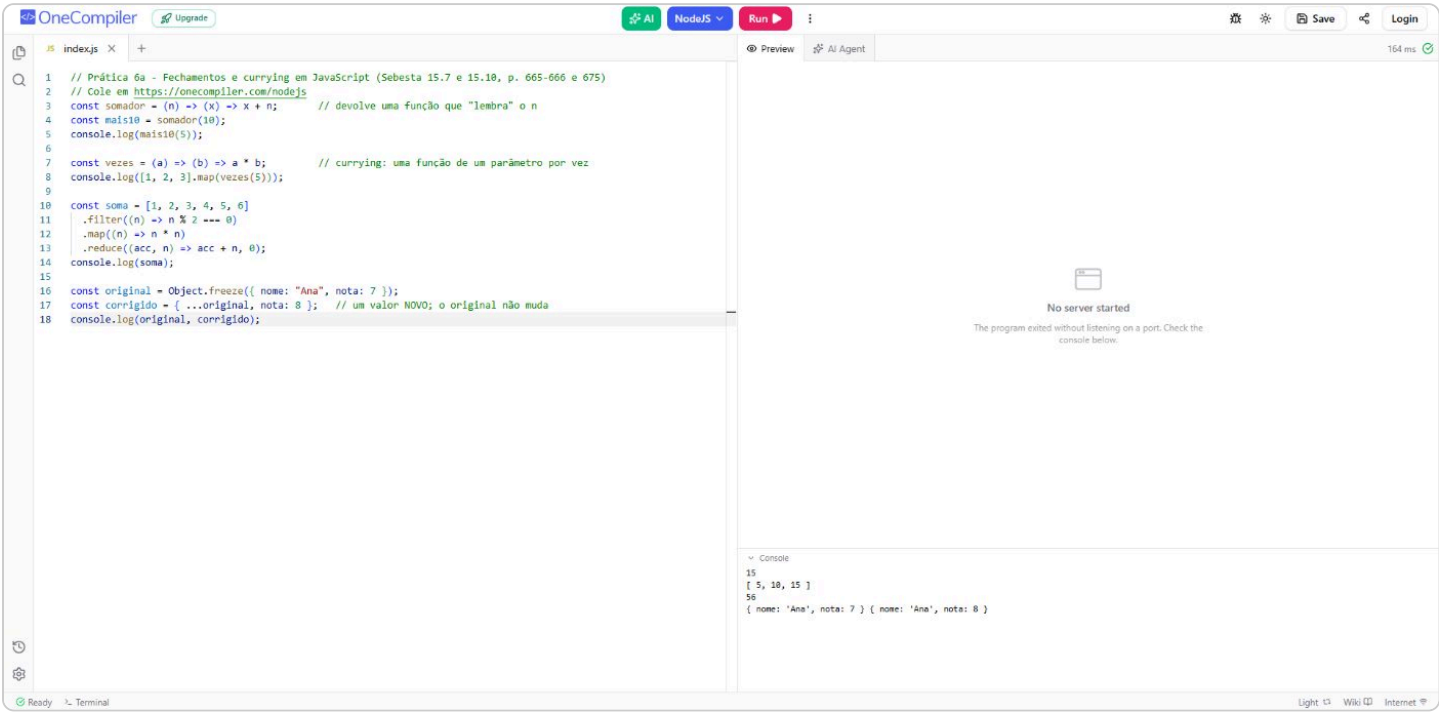
Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) imprime 15. [1, 2, 3].map(vezes(5)) imprime [5, 10, 15].

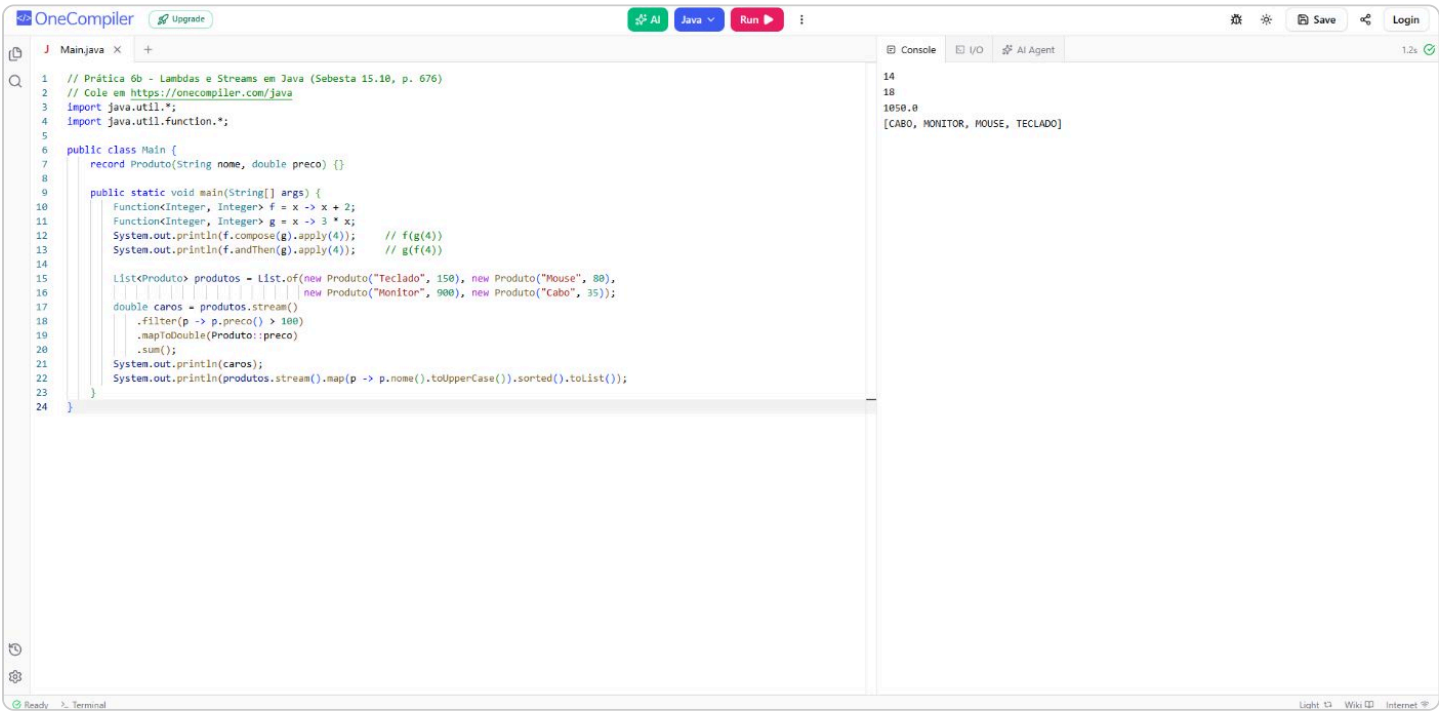
Saída do JavaScript

não respondido



Saída do Java

não respondido



6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

A função devolvida fecha sobre o ambiente léxico em que foi criada e mantém uma referência ao parâmetro n=10. Esse fechamento continua acessível mesmo depois que somador(10) terminou.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

compose aplica primeiro g e depois f; andThen aplica primeiro f e depois g. compor(f,g)(4) significa f(g(4)), portanto corresponde a f.compose(g).

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
Exercício 1 (Java):
int soma = nums.stream()
    .filter(n → n % 2 == 0)
    .mapToInt(n → n * n)
    .sum();
System.out.println(soma);
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
Exercício 3 (Python):
com_desconto = [p * 0.9 if p > 100 else p for p in precos]
print(com_desconto)
```

Saídas das suas versões

```
Exercício 1 (Java): 244

Exercício 3 (Python): [50.0, 108.0, 80.0, 180.0]
(iguais às saídas das versões originais com laço)
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional é mais concisa para filtrar e transformar coleções. A versão com laço pode ser mais fácil de depurar passo a passo, pois permite inspecionar cada iteração; pipelines também podem ser depurados observando cada etapa.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell é mais declarativa: mostra diretamente gerar a lista, elevar ao cubo e somar, sem explicitar contador e acumulador. Aceitaria custo extra em processamento offline, análise de dados ou tarefas não críticas; evitaria em sistemas de tempo real ou com limites rígidos de latência.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Surpreendeu-me que Haskell consiga trabalhar com listas infinitas calculando apenas a parte necessária, e que funções puras sejam mais fáceis de testar.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro é a variável de compreensão escrita incorretamente no filtro: a lista menor deve usar o mesmo nome de variável introduzido após <-. A forma correta é sort (h:t) = sort [b | b <- t, b <= h] ++ [h] ++ sort [b | b <- t, b > h]. O compilador indica que a expressão inferida não corresponde ao tipo esperado para a lista/elemento.